

IF-оператор

Използва се за проверка на логическо условие.

```
IF <условие>
THEN <заявки>;
ELSE <заявки>;
END IF;
```

Пример:

```
DELIMITER $$
CREATE PROCEDURE check_even_or_odd(IN num INT)
BEGIN
    DECLARE result VARCHAR(20);

    -- Използваме IF за проверка дали числото е четно или нечетно
    IF num % 2 = 0 THEN
        SET result = 'Четно';
    ELSE
        SET result = 'Нечетно';
    END IF;

    -- Извеждаме резултата
    SELECT result AS "Резултат";
END $$
DELIMITER ;
```

IF конструкцията може да бъде използвана както в процедури, така и директно в SQL заявки или други контексти, като например:

1. **В SQL заявки** – можете да използвате **IF** конструкцията в SQL изрази, за да вземете решение в зависимост от дадено условие.
2. **В селективни изрази** – може да бъде използвана в комбинация с **CASE** или **IF()** за условно изпълнение на различни стойности в една и съща заявка.

В **SELECT** или **UPDATE** изрази, **IF** се използва като функция, и не се нуждае от допълнителни ключови думи като BEGIN или END.

```
IF(condition, true_value, false_value)
```

Пример 1:

```
SELECT name,  
IF(grade >= 5, 'Добър', 'Слаб') AS result  
FROM Students;
```

За всеки ред:

- ако grade >= 5 → "Добър"
- иначе → "Слаб"

Пример 2:

```
UPDATE Students  
SET status = IF(grade >= 5, 'Минал', 'Скъсан');
```

За всеки ред:

- сменя стойността според условието

IF (функция)	IF (конструкция)
В SELECT	В процедури
Връща стойност	Изпълнява код
1 команда	Много команди
Няма END	Има END IF

CASE – оператор

Операторът **CASE** в MySQL е много полезен за условно изпълнение на изрази в SQL заявки. Той позволява да се извършат различни действия в зависимост от резултата от дадено условие, подобно на конструкциите if-else и switch-case в програмните езици.

Видове CASE оператори

Има два основни типа CASE оператори:

1. **Simple CASE** – Сравнява дадена стойност с различни възможности.
2. **Search CASE** – Оценява различни условия с помощта на логически изрази.

1. Simple CASE

Синтаксис:

```
CASE <израз>
  WHEN <стойност1> THEN <результат1>
  WHEN <стойност2> THEN <результат2>
  ...
  ELSE <результат по подразбиране>
END
```

Тук операторът CASE започва с израз, който се сравнява с различни стойности (например стойност1, стойност2 и т.н.). Когато изразът съвпадне с една от стойностите, се изпълнява съответният резултат.

Пример:

```
SELECT name,
  CASE status
    WHEN 'active' THEN 'User is active'
    WHEN 'inactive' THEN 'User is inactive'
    ELSE 'Unknown status'
  END AS status_description
FROM users;
```

В този пример, за всяко име в таблицата users, в зависимост от стойността на полето status, се връща описанието на статуса.

2. Search CASE

Синтаксис:

```
CASE
  WHEN <условие1> THEN <результат1>
  WHEN <условие2> THEN <результат2>
  ...
  ELSE <результат по подразбиране>
END
```

Тук вместо да се сравняват стойности, се задават условия (логически изрази), които трябва да бъдат изпълнени.

Пример:

```
SELECT name,  
CASE  
    WHEN age < 18 THEN 'Underage'  
    WHEN age BETWEEN 18 AND 64 THEN 'Adult'  
    ELSE 'Senior'  
END AS age_group  
FROM people;
```

В този пример се проверява възрастта на всеки човек и се връща съответната група възраст (напр. "Underage", "Adult" или "Senior").

name	age
John	17
Alice	35
Bob	72
Sarah	60

name	age	age_group
John	17	Underage
Alice	35	Adult
Bob	72	Senior
Sarah	60	Adult

Simple CASE	Search CASE
сравнява стойности	проверява условия
само =	всякакви оператори
по-ограничен	по-гъвкав

Използване на CASE в SELECT заявки

Често използваме **CASE** в **SELECT** изрази за създаване на условни колони в резултатите от заявките.

Пример 1:

```
SELECT product_name, price,  
CASE  
    WHEN price > 100 THEN 'Expensive'  
    WHEN price BETWEEN 50 AND 100 THEN 'Moderate'  
    ELSE 'Cheap'  
END AS price_category  
FROM products;
```

Тук за всеки продукт се присвоява категория според цената.

product_name	price
Product A	150
Product B	75
Product C	25
Product D	200

product_name	price	price_category
Product A	150	Expensive
Product B	75	Moderate
Product C	25	Cheap
Product D	200	Expensive

Използване на CASE в UPDATE заявки

CASE може да се използва и в **UPDATE** заявки за обновяване на стойности в зависимост от условията.

Пример:

```
UPDATE students
SET grade = CASE
    WHEN grade < 60 THEN grade + 5
    WHEN grade BETWEEN 60 AND 80 THEN grade + 3
    ELSE grade + 1
END;
```

В този пример актуализираме оценките на студентите според тяхната стойност.

student_name	grade
John	65
Alice	85
Bob	92
Sarah	55

student_name	grade
John	70
Alice	88
Bob	93
Sarah	60

Използване на CASE в ORDER BY

CASE може да се използва и за сортиране на резултатите от заявка въз основа на определени условия.

Пример:

```

SELECT name,
       experience,
       CASE
         WHEN experience < 5 THEN 'Junior'
         WHEN experience BETWEEN 5 AND 10 THEN 'Mid-level'
         ELSE 'Senior'
       END AS experience_level
FROM employees
ORDER BY
  CASE
    WHEN experience < 5 THEN 1
    WHEN experience BETWEEN 5 AND 10 THEN 2
    ELSE 3
  END;

```

В този пример използваме **CASE** в **ORDER BY** за да сортираме служителите според нивото на опит:

- "Junior" (по-малко от 5 години опит),
- "Mid-level" (между 5 и 10 години опит),
- "Senior" (повече от 10 години опит).

Резултатът е сортиран първо по групата "Junior", после "Mid-level", и накрая "Senior".

name	experience
John	3
Alice	8
Bob	1
Sarah	15

name	experience	experience_level
Bob	1	Junior
John	3	Junior
Alice	8	Mid-level
Sarah	15	Senior

WHILE-цикъл

```

WHILE <условие> DO
<заявки>;
END WHILE;

```

Пример:

```
DELIMITER $$  
  
CREATE PROCEDURE sum_numbers()  
  
BEGIN  
  
    DECLARE sum INT DEFAULT 0;  
  
    DECLARE i INT DEFAULT 1;  
  
    WHILE i <= 10 DO  
  
        SET sum = sum + i;  
  
        SET i = i + 1;  
  
    END WHILE;  
  
    SELECT sum AS total_sum;  
  
END$$  
  
DELIMITER ;
```

REPEAT цикъл в MySQL е друг вид цикъл, който работи по подобен начин на **WHILE цикъл**, но има малка разлика в начина на изпълнение. В **REPEAT цикъл**, условието за спиране на цикъла се проверява **след** всяка итерация, а не преди нея. Това означава, че тялото на цикъла ще се изпълни поне веднъж, дори ако условието за спиране не е изпълнено.

Синтаксис на REPEAT цикъл:

```
REPEAT  
  
    -- Вашият код тук  
  
UNTIL условие  
  
END REPEAT;
```

- **условие:** Това е логическо условие, което трябва да бъде **неистинско**, за да продължи цикълът. Когато условието стане **истинско**, цикълът ще спре.

Как работи:

1. **Тялото на цикъла** се изпълнява поне веднъж, преди да бъде проверено условието за спиране.
2. След всяка итерация, условието в UNTIL се проверява, и ако е **истинско**, цикълът приключва. Ако е **неистинско**, цикълът продължава.

Пример 1: Използване на REPEAT цикъл за изчисляване на сумата на числата от 1 до 10

```
DELIMITER $$  
  
CREATE PROCEDURE sum_numbers_repeat()  
  
BEGIN  
  
    DECLARE sum INT DEFAULT 0;  
  
    DECLARE i INT DEFAULT 1;  
  
    REPEAT  
  
        SET sum = sum + i;  
  
        SET i = i + 1;  
  
    UNTIL i > 10  
  
    END REPEAT;  
  
    SELECT sum AS total_sum;  
  
END$$  
  
DELIMITER ;
```

Извикване на процедурата:

```
CALL sum_numbers_repeat();
```

LOOP

LOOP в MySQL е конструкция за създаване на **безкраен цикъл**, който изпълнява определен код многократно, докато не бъде прекъснат с команда като **LEAVE** или **ITERATE**. Тази конструкция се използва най-често в **съхранени процедури** и позволява да се повтори блок от код многократно въз основа на определени условия или докато не се срещне изключение.

Как работи конструкцията LOOP?

1. **LOOP** започва изпълнение на код в тялото на цикъла.
2. Тялото на цикъла може да съдържа условия или логика, която да прекрати цикъла или да го накара да продължи.
3. **LEAVE** се използва за излизане от цикъла, а **ITERATE** за пропускане на текущата итерация и преминаване към следващата.
4. Цикълът може да бъде прекратен, когато достигне желаното условие за изход.

Синтаксис на LOOP:

LOOP

```
-- Вашият SQL код тук

IF <условие за изход> THEN

    LEAVE <име_на_циклата>;

END IF;

END LOOP;
```

Пример за използване на LOOP:

Изпълнение на цикъл, който изчислява факториел

Нека разгледаме пример за **изчисляване на факториел** на дадено число с помощта на **LOOP** в съхранена процедура.

```
DELIMITER $$

CREATE PROCEDURE calculate_factorial(IN num INT)

BEGIN

    DECLARE result INT DEFAULT 1;

    DECLARE counter INT DEFAULT 1;

    loop_start: LOOP -- Стартиране на цикъла

        IF counter > num THEN

            LEAVE loop_start; -- Прекратява цикъла, ако сме достигнали желанния резултат

        END IF;

        SET result = result * counter;

        SET counter = counter + 1;

    END LOOP;

    SELECT result AS factorial; -- Показваме резултата

END$$

DELIMITER ;
```

Извикване на процедурата:

```
CALL calculate_factorial(5);
```

Използване на LOOP с LEAVE и ITERATE

В следващия пример ще разгледаме как да използваме **LEAVE** и **ITERATE** вътре в **LOOP**, за да управляваме по-гъвкаво логиката на цикъла.

```

DELIMITER $$

CREATE PROCEDURE loop_example()

BEGIN

    DECLARE counter INT DEFAULT 1;

    DECLARE max_count INT DEFAULT 10;

    -- Цикъл, който преминава през числата от 1 до 10

    loop_start: LOOP

        IF counter = 5 THEN

            SET counter = counter + 1;

            ITERATE loop_start; -- Пропуска текущата итерация и преминава към следващото

        END IF;

        IF counter > max_count THEN

            LEAVE loop_start; -- Прекратява цикъла, когато броячът стане по-голям от max_count

        END IF;

        SELECT counter; -- Показваме текущия брояч

        SET counter = counter + 1;

    END LOOP;

END$$

DELIMITER ;

```

Извикване на процедурата:

```
CALL loop_example();
```

Временни таблици

Временните таблици в MySQL са специални таблици, които съществуват само за времето на текущата сесия или докато не бъде изрично изтрита. Те се използват за временно съхранение на данни, които са нужни само за определена операция и не трябва да се запазват в базата данни за постоянно.

За какво се използват временни таблици?

1. **Предоставяне на междинни резултати:** можете да използвате временни таблици за съхранение на междинни резултати, които ще ви трябват само по време на конкретната сесия. Това е полезно при сложни заявки с много подзаявки или когато работите с много данни, които не искате да задържате в основната база данни.
2. **Оптимизация на заявки:** често временно съхранение на данни може да подобри производителността на някои сложни операции, като обединяване на много таблици или при работа с големи обеми данни.

3. **Изолиране на изчисления:** когато искате да работите с данни, без да влияете на основните таблици в базата.
4. **Променливи за заявки с многократни стъпки:** временните таблици могат да се използват за задържане на резултати от частични изчисления, като например при обработка на поредица от данни или при работа с потоци от резултати.

Как да създадем и използваме временни таблици?

Създаване на временна таблица

Временната таблица се създава със същия синтаксис като обикновената таблица, но със специален **TEMPORARY** ключовата дума. Тя ще бъде достъпна само в рамките на сесията, в която е създадена.

```
CREATE TEMPORARY TABLE temp_table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
);
```

Създаване и използване на временна таблица

```
CREATE TEMPORARY TABLE temp_employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    salary DECIMAL(10, 2)  
);  
  
INSERT INTO temp_employees (id, name, salary)  
VALUES (1, 'John Doe', 50000),  
        (2, 'Jane Smith', 60000),  
        (3, 'Sam Brown', 45000);  
  
SELECT * FROM temp_employees;
```

Тази временна таблица ще съдържа данни за служителите и ще бъде достъпна само докато сесията не бъде прекратена или таблицата не бъде изтрита.

Изтриване на временна таблица

Временните таблици могат да бъдат изтрети по два начина:

1. Когато сесията завърши (MySQL автоматично изтрива временната таблица).
2. Чрез изрично изтриване:

```
DROP TEMPORARY TABLE temp_employees;
```

Използване на временна таблица в процедури

Временните таблици често се използват в съхранени процедури за съхранение на междинни резултати. Например:

```
DELIMITER $$  
  
CREATE PROCEDURE calculate_salary_increase()  
  
BEGIN  
  
    CREATE TEMPORARY TABLE temp_salaries (  
        id INT,  
        current_salary DECIMAL(10, 2),  
        new_salary DECIMAL(10, 2)  
    );  
  
    INSERT INTO temp_salaries (id, current_salary)  
    SELECT id, salary FROM employees;  
  
    UPDATE temp_salaries  
SET new_salary = current_salary * 1.10; -- увеличаваме с 10%  
  
    SELECT * FROM temp_salaries; -- показваме новите заплати  
  
    DROP TEMPORARY TABLE temp_salaries; -- премахваме временната таблица  
  
END$$  
  
DELIMITER ;
```

Тази процедура създава временна таблица за съхранение на новите заплати на служителите след увеличението.

Предимства на временните таблици:

1. **Изолираност:** данните в временната таблица не се смесват с данните в постоянните таблици.
2. **Автоматично изтриване:** след края на сесията, временните таблици се изтриват автоматично, което означава, че не трябва да се притеснявате за тяхното управление.
3. **Подобрена производителност:** за някои сложни операции или големи заявки временните таблици могат да помогнат за по-добро организиране на данни и намаляване на натоварването на системата.

Ограничения:

1. **Живот на сесията:** Временните таблици съществуват само докато сесията е активна или докато не бъдат изтрити.
2. **Изолираност на сесията:** Временни таблици са достъпни само за текущата сесия. Не могат да бъдат достъпвани от други сесии.
3. **Не може да се използва в индекси и външни ключове:** Не можете да създавате външни ключове или индекси в тях, както и да ги използвате в транзакции с дълъг живот.

Временните таблици са полезен инструмент за работа с данни, които не трябва да се съхраняват постоянно в базата. Те могат да подобрят производителността при сложни заявки или процедури, като осигуряват временно пространство за съхранение на резултати и намаляват натоварването върху основните таблици в базата данни.

Курсори

Курсор в MySQL е механизъм за работа с резултатите от SQL заявки по начин, който позволява ред по ред обработка на данни. Въпреки че SQL обикновено обработва множество редове едновременно, курсорите позволяват на разработчика да обработва всеки ред от резултатите поотделно.

Това е особено полезно, когато трябва да изпълняваш логика или да извършваш операции за всеки ред от даден резултат.

Как работи курсорът?

Курсорите работят чрез следните стъпки:

1. **Деклариране на курсор** - създаване на курсор за заявка.

```
DECLARE cursor_name CURSOR FOR  
SELECT column1, column2 FROM table_name;
```

- cursor_name – име на курсора.
- SELECT column1, column2 FROM table_name; – заявка, която връща резултатния набор.

2. **Отваряне на курсор** - изпълняване на заявката и събиране на резултатите.

```
OPEN cursor_name;
```

3. **Извличане на редове** - извличане на данни по редове от резултатите на заявката.

```
FETCH cursor_name INTO variable1, variable2, ...;
```

- cursor_name – името на курсора, от който извличаме данни.
- variable1, variable2, ... – променливите, в които ще се запазят стойностите на извлечения ред.

*Ако извличаме няколко колони, броят на променливите **трябва да съвпада** с броя на колоните.*

4. **Затваряне на курсор** - освобождаване на ресурсите след обработката.

```
CLOSE cursor_name;
```

Кога се използва курсор?

Курсорите се използват в случаи, когато е необходимо да се обработват данни ред по ред, например:

- За извършване на сложни операции върху всеки ред в резултатите от заявка.
- Когато трябва да направиш обновления или изтривания на база на стойности от множество таблици.
- Когато трябва да изчислиш нещо по редовете в резултата на заявката.

Пример за използване на курсор в MySQL

Нека разгледаме един прост пример, в който използваме курсор за извличане и обработка на данни от таблица:

Да кажем, че имаме таблица с имената на служителите и заплатите им.

Сега ще създадем процедура, която използва курсор за обработка на заплатите на служителите.

Примерен код за съхранена процедура с курсор:

```
DELIMITER $$

CREATE PROCEDURE process_employee_salaries()

BEGIN

    DECLARE done INT DEFAULT 0;

    DECLARE emp_name VARCHAR(100);

    DECLARE emp_salary DECIMAL(10,2);

    DECLARE emp_cursor CURSOR FOR

        SELECT name, salary FROM employees;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN emp_cursor; -- Отваряме курсора

    read_loop: LOOP -- Извличаме редове един по един

        FETCH emp_cursor INTO emp_name, emp_salary;

        IF done THEN

            LEAVE read_loop; -- Прекратяване на цикъла ако няма повече редове

        END IF;

        -- Извършваме някаква обработка (например увеличаване на заплатата с 10%)

        SET emp_salary = emp_salary * 1.10;

        -- Актуализираме заплатата в таблицата

        UPDATE employees SET salary = emp_salary WHERE name = emp_name;

        -- Може да добавим и логика за извеждане на резултат или други действия

        SELECT emp_name AS employee_name, emp_salary AS new_salary;

    END LOOP;

    CLOSE emp_cursor; -- Затваряме курсора

END$$

DELIMITER ;
```

Когато извикаш процедурата, за всеки служител ще се изпълни операцията за увеличение на заплатата и ще видим новата заплата на всеки служител в резултатите от SELECT заявката.

Курсорите в MySQL са полезни, когато трябва да обработваш ред по ред данни от таблици, особено когато се налага да извършваш сложни операции върху всеки ред. Въпреки това, трябва да се внимава с тях, тъй като те могат да доведат до намаляване на производителността при обработка на големи

набори от данни. В такива случаи е препоръчително да се използват алтернативни методи като **batch processing** или обработка чрез SQL заявки.

Грешки при съхранени процедури и използване

на INSERT...ON DUPLICATE KEY UPDATE

При създаване и изпълнение на съхранени процедури в MySQL е важно да се предвиди механизъм за обработка на грешки. Това включва вземане на решение дали изпълнението на процедурата трябва да продължи или да бъде прекратено, както и осигуряване на подходящи съобщения за възникналите грешки. Обработката на грешки е от ключово значение за поддържане на коректността на данните и избягване на неочаквани състояния в базата.

Обработка на грешки с MySQL HANDLER-и

MySQL предоставя механизъм за обработка на грешки чрез HANDLER-и. Те позволяват прихващане както на общи, така и на специфични грешки, което позволява по-гъвкав контрол върху изпълнението на съхранените процедури.

Деклариране на HANDLER:

В MySQL **DECLARE HANDLER** трябва да се декларира **вътре в блока BEGIN ... END** на съхранената процедура, но **след всички декларации на променливи** и преди основния код на процедурата.

Важното е, че **HANDLER-ът трябва да се декларира след всички променливи**, защото MySQL изисква декларациите да са в определен ред:

1. **Променливи** (DECLARE variable_name ...)
2. **HANDLER-и** (DECLARE CONTINUE HANDLER FOR ...)
3. **Курсори** (ако има)
4. Основен код

Ако декларацията на HANDLER бъде поставена **преди променливите**, ще получите грешка.

```
DECLARE action HANDLER FOR condition_value statement;
```

- action може да бъде:
 - CONTINUE – Продължава изпълнението на текущия блок (BEGIN ... END), игнорирайки грешката.
 - EXIT – Прекратява изпълнението на текущия блок и излиза от него.
- condition_value може да бъде:
 - Код за грешка в MySQL (напр. 1051 за неоткрита таблица).
 - SQLSTATE стойност (напр. '42S01' за вече съществуваща таблица).
 - Общи грешки като SQLWARNING, NOTFOUND, SQLEXCEPTION.

```

DELIMITER //

CREATE PROCEDURE exampleProc()
BEGIN
    -- Деклариране на променливи
    DECLARE finished INT DEFAULT 0;

    -- Деклариране на HANDLER за прихващане на грешка
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;

    -- Основен код на процедурата
    WHILE finished = 0 DO
        -- Код за изпълнение
    END WHILE;
END //

DELIMITER ;

```

Често срещани грешки в MySQL

Код за грешка	SQLSTATE	Описание
1051	42S02	Таблицата не съществува
1062	23000	Дублираща се уникална стойност
1146	42S02	Таблицата не е намерена
1216	23000	Нарушена релационна връзка (foreign key constraint)
1217	23000	Изтриване на запис с релационна зависимост
1364	HY000	Липсва стойност за NOT NULL колона
1451	23000	Не може да се изтрие родителски запис (foreign key constraint)
1452	23000	Вмъкване или актуализиране с несъществуващ външен ключ
2002	HY000	Неуспешно свързване към MySQL сървър

Пример за обработка на грешка:

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '42S01' SELECT 'SQLSTATE 42S01 occurred';  
DECLARE CONTINUE HANDLER FOR NOTFOUND SELECT 'NOT FOUND exception occurred';
```

Този подход позволява на процедурата да продължи изпълнението си, дори ако срещне конкретни грешки, като в същото време предоставя диагностична информация.

Прекъсване на кодов блок с LEAVE

LEAVE се използва за напускане на етикетиран блок (BEGIN ... END). Това е полезно в ситуации, в които искаме да прекратим изпълнението на даден кодов сегмент при определено условие, без да излизаме от цялата процедура.

Пример за използване на LEAVE:

```
DELIMITER $$  
  
CREATE PROCEDURE testProc(IN param INT)  
  
out_block: BEGIN  
    DECLARE res INT;  
    SET res = param;  
  
    inner_block: BEGIN  
        IF (res = 1) THEN  
            LEAVE inner_block;  
        END IF;  
  
        SELECT 'Executed only if param is 0';  
    END inner_block;  
  
    SELECT 'End of program';  
END out_block;  
  
$$  
DELIMITER ;
```

При извикване на процедурата с 1, блокът inner_block ще бъде пропуснат.

INSERT...ON DUPLICATE KEY UPDATE

Това е механизъм, който позволява вмъкване на запис в база данни, като при дублиращ се уникален ключ записът не предизвиква грешка, а се актуализира. Този механизъм може да бъде свързан с грешки като 1062 (Duplicate entry) или нарушения на UNIQUE и PRIMARY KEY. ON DUPLICATE KEY UPDATE не позволява дублиране на уникалния ключ. Вместо това, ако има конфликт, той актуализира

съществуващия запис със зададените стойности. Така се избягва грешката Duplicate entry for key, която обикновено се получава при опит за вмъкване на запис с вече съществуваща уникална стойност.

Пример за използване:

```
INSERT INTO salarypayments(`coach_id`, `month`, `year`, `salaryAmount`, `dateOfPayment`)
VALUES (1, 1, 2016, 1800, NOW())
ON DUPLICATE KEY UPDATE
    salaryAmount = salaryAmount + 1800,
    dateOfPayment = NOW();
```

Ако записът (1, 1, 2016) не съществува, ще бъде добавен като нов. Ако вече съществува, ще бъде актуализиран чрез увеличаване на salaryAmount с 1800 и обновяване на dateOfPayment.

Обработка на грешки и управление на транзакции

Важно е да се подготвите за грешки при работа със съхранени процедури, особено когато използвате транзакции. Често при работа с транзакции трябва да се гарантира, че в случай на грешка, всички предишни операции в рамките на транзакцията ще бъдат отменени (rollback). Това може да се постигне чрез използване на DECLARE CONTINUE HANDLER.

Пример за транзакция с обработка на грешки:

```
START TRANSACTION;
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
    -- Обработваме грешката и правим rollback на транзакцията
    ROLLBACK;
    SELECT 'Transaction rolled back due to error';
END;
-- Вашите SQL изрази
INSERT INTO table_name VALUES (...);
UPDATE another_table SET ...;
COMMIT;
```

1. Ако възникне SQL грешка, обработчикът ще се активира и ще изпълни командата **ROLLBACK**, за да върне транзакцията назад.
2. **ROLLBACK**: Отменя всички действия в транзакцията, ако възникне грешка.
3. **INSERT INTO table_name VALUES (...);** и **UPDATE another_table SET ...;**: Това са SQL изрази, които може да генерират грешка по време на изпълнение.
4. **COMMIT**: Ако няма грешка, транзакцията приключва и всички промени се потвърдиха.

Ако има грешка, всички промени в транзакцията се отменят, а ако няма грешка, всички действия се записват в базата данни с **COMMIT**.

Логическа обработка на грешки

Извън стандартните грешки на SQL, можете да добавите и логическа обработка на специфични условия. Например, при обработка на бизнес логика може да се налага да проверявате специфични условия и да изпълнявате различни действия в зависимост от тях.

Пример за логическа обработка на грешки:

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '42S02'

BEGIN

  -- В случай на грешка за липсваща таблица

  SELECT 'Table not found, proceeding with alternative logic';

  -- Алтернативен код или корекция

END;

-- Основен код

SELECT * FROM non_existent_table;
```

Примерът показва как може да се обработи конкретна грешка, без да се прекъсва изпълнението на процедурата. Ако в основния код възникне грешка, която съответства на дефинираната SQLSTATE стойност ('42S02' в случая), тогава се изпълнява обработчикът и изпълнението продължава.

Заклучение

Грешките в MySQL могат да варират от малки проблеми (като опити за добавяне на дублиращи се записи) до сериозни проблеми с транзакциите и зависимостите между таблиците. Затова е важно да се използват механизми за обработка на грешки, които не само да улавят грешките, но и да позволяват на процедурата да продължи изпълнението си или да прекрати изпълнението на блока по подходящ начин.